

SOFTWARE TESTING · AI-FIRST · REDESIGNED (2026)

Automation & Regression Testing

What to automate, how to structure it, and how AI now writes, heals and runs the tests for you.

Regression

Frameworks

Tools 2026

AI-first & agentic

IT Faculty, HCMUS · examples in Playwright · Selenium · Cypress

Test automation in 5W1H

Six questions frame the whole deck — What, Why, When, Where, Who, How.

WHAT

Software runs & checks the tests — repeatably, unattended.

WHY

Speed, coverage, repeatability — and a CI merge gate.

WHEN

Stable, repeated, regression-prone tests — sprint to release.

WHERE

Web · mobile · desktop · API — and cloud device farms.

WHO

SDETs, developers, the whole team — and now AI agents.

HOW

Frameworks (POM/BDD), the 3 A's, in CI — increasingly AI-assisted.

What is test automation — and why

Software runs the tests, checks the results, and reports — repeatably, at machine speed.

WHAT

- › Code drives the app & asserts results.
- › Replaces slow, error-prone manual reruns.
- › Runs on every change, unattended.
- › Gives a fast pass/fail signal.

WHY

- › Speed & repeatability at scale.
- › Wider coverage than manual can reach.
- › Frees humans for exploratory testing.
- › Enables CI/CD — the merge gate.

What to automate — and what not to

Automation is an investment. Spend it where it pays back — examples on the EShop app.

AUTOMATE

- › **Repetitive / regression-prone** — e.g. login & checkout smoke, every build
- › **High-risk, business-critical** — e.g. payment & order-total math
- › **Data-driven (many inputs)** — e.g. 20 coupon codes across carts
- › **Stable, mature features** — e.g. product search & add-to-cart

DON'T (yet) AUTOMATE

- › **One-off / rarely run** — e.g. a one-time data-migration check
- › **Unstable, changing UI** — e.g. checkout redesign in weekly flux
- › **Exploratory & usability** — e.g. first-impression UX of a new flow
- › **Needs human judgement** — e.g. “is the banner image appealing?”

RULE OF THUMB automate what is repeated, stable and objective; leave the one-off, fluid and subjective to humans.

WHEN

When to automate — timing matters

The right test, at the right point in the lifecycle — or the ROI never arrives.

WHEN IN THE LIFECYCLE

- › **Shift-left** — unit & API tests from day one.
- › **After it stabilises** — automate UI once it stops churning.
- › **Before every release** — the regression suite.
- › **Continuously** — on every commit, in CI.

WHEN IT PAYS OFF

- › The test is run many times (regression).
- › Manual is slow, tedious, error-prone.
- › Requirements are stable & well understood.
- › Automation cost < repeated manual cost.

Who does test automation

From a lone engineer to the whole team — and, increasingly, AI agents.

SDET / Automation engineer

Designs the framework; writes & maintains the suites; owns CI.

Developers

Unit & API tests; build for testability; fast feedback (shift-left).

Manual / QA testers

Author codeless tests; do the exploratory testing machines can't.

Whole team + AI

'Whole-team quality'; AI agents draft, run, heal — humans review.

The automation pyramid

Automate many fast checks low down; keep the slow, brittle UI layer thin.

UI / E2E — few — slow, brittle, whole-system

API / integration — more — fast, stable, business logic

Unit — most — instant, pinpoint failures

Anti-pattern: the ice-cream cone — mostly slow UI tests. It is expensive, flaky, and slow to run.

Regression testing — what & why

Re-verify that a change didn't break what already worked. The classic job for automation.

- › **The problem** — every fix or feature can silently break an unrelated part.
- › **The job** — re-run existing tests after each change to catch that.
- › **Why it's hard manually** — suites grow huge; nobody re-runs them all by hand.
- › **So we automate it** — the machine re-runs the whole suite on every push.

Retesting vs regression testing

Two different jobs people constantly confuse.

	Retesting	Regression testing
Purpose	confirm a specific bug is fixed	confirm nothing else broke
Scope	only the failed test case	the surrounding / whole suite
Inputs	the same data that failed	broad, varied inputs
Automation	usually manual, targeted	the prime candidate to automate

Three ways to choose what to re-run

Re-running everything gets too slow. Pick a smarter strategy.

Procedural

Re-run a fixed suite every time.

simple, safe, but slow as it grows

Risk-based

Prioritise by risk & change impact.

focus effort where breakage hurts

Test impact analysis

Run only the tests a diff can affect.

AI/graph picks the subset — minutes, not hours

Test impact analysis — run only what matters

Don't re-run 10,000 tests for a one-line change; run the ones it can break.

EXAMPLE — select by changed files

TEST TECHNIQUE

- › **Map tests → code** — learn which tests exercise which files.
- › **Select on a diff** — pick the subset the change can affect.
- › **Run those first** — fail fast; defer the rest to nightly.
- › **Payoff** — minutes per PR instead of hours.

```
# only run tests touching the changed code
git diff --name-only main...HEAD
-> src/checkout/coupon.ts

# impact tool maps that file to its tests
npx jest --findRelatedTests \
  src/checkout/coupon.ts
# tools: Launchable (CloudBees) · Seallights ·
#       Jest --findRelatedTests · Nx affected
```

TREND Selective regression is the frontier — keep the merge gate fast as the suite grows.

Every automated test = 3 A's

Arrange the object, Act on it, Assert the outcome. Keep them in that order.

1

Arrange

Find/identify the element and set up the known state.

locator + fixtures

2

Act

Perform the user action — click, type, submit, call.

the single behaviour under test

3

Assert

Check the outcome against the expected result.

one clear, meaningful check

Automation framework patterns

How you organise tests decides how well they scale and survive change.

Pattern	Idea · when to use
Linear (record-playback)	one script per case · quick demos, worst to maintain
Modular	reusable functions per screen/component · medium suites
Data-driven	one script, many data rows · same flow, many inputs
Keyword-driven	actions as keywords in a table · non-coders author cases
BDD (Given/When/Then)	Gherkin + steps · shared language with the business
Hybrid	combine the above · what most real suites become

Page Object Model & the DRY principle

Separate 'what a page can do' from 'what a test checks' — so one UI change = one edit.

EXAMPLE — Playwright + POM

TEST TECHNIQUE

- › **Page Object** — one class per page; locators & actions live there.
- › **DRY** — never repeat a selector; define it once, reuse everywhere.
- › **Readable tests** — the test reads like the user's intent, not CSS.
- › **Screenplay** — actors + tasks; scales POM for large suites.

```
// login.page.ts — the Page Object
class LoginPage {
  email = () => page.getByLabel("Email");
  submit = () => page.getByRole("button",
    { name:"Sign in" });
}
// the test reads like intent:
await login.email().fill("a@x.vn");
await login.submit().click();
```

WHY A selector changes in one place — not scattered across 200 tests.

Flaky tests — the trust killer

A test that passes and fails on the same code trains the team to ignore red.

Timing

races, fixed sleeps, no auto-wait → use web-first assertions.

Ordering

tests share state; pass alone, fail in a suite → isolate & reset.

Environment

CPU load, network, test data drift → pin & mock dependencies.

The fix

auto-wait, independent tests, retries as a signal — not a cure.

GUI automation is programming

Treat it like production code, or it becomes a flaky, expensive liability.

- › **It's software** — design, review, refactor and version it like real code.
- › **It pays off late** — upfront cost is high; ROI comes from many reruns.
- › **Brittle by nature** — UI changes break selectors; POM + auto-wait mitigate.
- › **Not everything** — don't automate what a human eye checks better.

Web automation tools — 2026

The market shifted fast: Playwright now leads new projects.

	Playwright	Selenium	Cypress
New-project adoption	45% (leading)	22% (declining)	14%
Languages	TS · Py · Java · C#	all major langs	JavaScript only
Auto-wait	yes, every action	manual waits	assertion retry
Parallelism	free, built-in	Grid / cloud	paid cloud
~7.4M vs 2.1M vs ~6M weekly npm downloads · source: 2026 framework surveys.			

Which framework — and when

No single winner; match the tool to the team and the app.

Pick Playwright

New projects; TS/Py/Java/C#; multi-tab & cross-origin; free parallel; AI test-gen.

Pick Cypress

JS-only team; single-origin SPA; love the time-travel debugger.

Pick Selenium

Ruby/Kotlin/other langs; stable legacy suites; Appium mobile; W3C standard.

Automation across platforms

One discipline — but drivers, locators and pain points differ per platform.

Web

browsers · the DOM · Playwright/Selenium/Cypress.

Mobile

native & hybrid · iOS + Android · Appium/Espresso/XCUITest.

Desktop

Windows/macOS native · UI Automation · WinAppDriver/FlaUI.

API / services

no UI · fast & stable · Postman/REST Assured.

Cloud device farms

real devices at scale · BrowserStack/Sauce/LambdaTest.

Cross-platform

one script, many targets · Appium · Playwright.

Mobile automation tools

Native vs hybrid; iOS and Android differ — run on real devices, not just emulators.

Appium

cross-platform (iOS+Android) · WebDriver standard · one API.

Espresso

Android native (Google) · fast, in-process, white-box.

XCUITest

iOS native (Apple) · fast · Xcode-integrated.

Maestro

modern · simple YAML flows · resilient (rising in 2025).

Detox

grey-box for React Native · sync-aware, less flaky.

Device farms

BrowserStack · Sauce Labs · LambdaTest · Kobiton · AWS.

A mobile automated test

Appium drives a real device via WebDriver; Maestro keeps the same flow dead simple.

EXAMPLE — Appium (JS) + Maestro (YAML)

TEST TECHNIQUE

- › **Accessibility id** — the stable, cross-platform locator.
- › **Real device / emulator** — via an Appium server or device farm.
- › **One API, two OSes** — same test, iOS & Android capabilities.
- › **Maestro alternative** — YAML flow, auto-wait, far less flaky.

```
// Appium + WebdriverIO
const btn = await driver.$("~loginBtn"); // a11y id
await btn.click();
await driver.$("~email").setValue("a@x.vn");
await expect(driver.$("~welcome")).toBeDisplayed();

# Maestro — flow.yaml (maestro test flow.yaml)
- launchApp
- tapOn: "Sign in"
- inputText: "a@x.vn"
- assertVisible: "Welcome"
```

TIP Locate by accessibility id — stable across iOS/Android, and what screen readers use.

Desktop automation tools

Windows & macOS native apps — object-based where possible, image-based as fallback.

Appium (Win/Mac)

WinAppDriver / Mac2 drivers · WebDriver standard · OSS.

FlaUI / White

(.NET) built on Windows UI Automation · object-based · OSS.

WinAppDriver

Microsoft's Windows driver · object-based · (maintenance slowed).

Ranorex / TestComplete

commercial · record-playback + AI object recognition.

SikuliX / Autot

image / pixel-based fallback when objects aren't exposed.

Playwright / Selenium

Electron & web-based desktop apps.

A desktop automated test

Object-based drivers read the native UI tree; fall back to image-based only when forced.

EXAMPLE — WinAppDriver (C#) + FlaUI

TEST TECHNIQUE

- › **AutomationId** — the desktop equivalent of a stable id.
- › **WinAppDriver** — WebDriver for Windows apps (Appium-compatible).
- › **FlaUI** — (.NET) drives UI Automation directly, no server.
- › **Image fallback** — SikuliX / AutoIt when no object is exposed.

```
// WinAppDriver — Appium-style, C#
session.FindElementByName("Seven").Click();
session.FindElementByName("Plus").Click();
session.FindElementByName("Eight").Click();
Assert.AreEqual("15", display.Text);

// FlaUI — UI Automation directly
var win = app.GetMainWindow(automation);
win.FindFirstDescendant(
    cf.ByAutomationId("btn7")).AsButton().Click();
```

TIP Prefer AutomationId; image-based automation is brittle — treat it as a last resort.

How automation differs by platform

Same 3 A's — very different plumbing, locators and failure modes.

Aspect	Web	Mobile	Desktop
Driver	WebDriver / CDP	Appium · Espresso · XCUITest	UIA · WinAppDriver · image
Locators	CSS · XPath · role	accessibility id · xpath	automation id · UIA · image
Environment	browsers	emulators + real devices	OS versions · native controls
Parallel run	easy, cheap	device farms	harder, VM-bound
Top challenge	cross-browser	device fragmentation	no stable object model

Other layers — API, visual & more

The same discipline also applies below and around the UI.

Mobile

Appium — the open standard (iOS/Android) · Espresso · XCUITest.

API

Postman/Newman · REST Assured · Playwright APIRequest — fast & stable.

Visual

Applitools / Percy — AI visual diffs catch what assertions miss.

Accessibility

axe-core in the pipeline — automate a11y checks.

Desktop

WinAppDriver · Playwright (Electron).

Contract

Pact — automate the API contract between services.

Codeless & low-code automation

Let non-coders author tests — and cut maintenance with smarter tooling.

- › **Natural-language tests** — testRigor / TestBooster: write a case in plain English.
- › **Record + refine** — Katalon · Selenium IDE · Playwright codegen scaffold scripts.
- › **Enterprise low-code** — Tricentis Tosca · mabl · Reflect · Testim.
- › **The trade-off** — fast to start; watch for vendor lock-in and opaque logic.

AI in test automation

AI now writes, maintains and stabilises the suite — you steer and review.

Self-healing locators

When the UI changes, AI re-finds the element instead of failing.

Testim · mabl · Healenium

Visual AI

AI compares screenshots, ignoring noise, flagging real changes.

Applitools · Percy

NL test generation

Describe intent; AI drafts the test to review.

testRigor · Playwright MCP

Agentic testing — from scripts to agents

Gartner now tracks 'Agentic Software Quality Assurance' — AI that tests, not just assists.

- › **Explore** — an agent crawls the app and proposes test cases from the UI/spec.
- › **Generate & run** — it writes, executes and reports the tests autonomously.
- › **Self-heal** — it repairs broken locators and updates flows as the app changes.
- › **MCP tools** — agents drive Playwright/browsers via MCP; you review the diffs.

STILL YOU IN THE LOOP Agents draft and heal; a human reviews intent, owns the assertions, and signs off.

A modern automated test

Auto-wait, web-first assertions and role locators make it stable and readable.

EXAMPLE — checkout happy path

TEST TECHNIQUE

- › **Role / label locators** — resilient to markup changes.
- › **Web-first assertions** — `expect()` auto-retries until true or times out.
- › **No manual sleeps** — auto-wait removes the classic flake source.
- › **Runs in parallel** — free, across browsers, in CI.

```
// Playwright + TypeScript
test("checkout with a coupon", async ({page}) => {
  await page.goto("/cart");
  await page.getByLabel("Coupon").fill("SAVE10");
  await page.getByRole("button",
    { name:"Apply" }).click();
  await expect(page.getByTestId("total"))
    .toHaveText("90.00"); // auto-retries
});
```

STABLE Role locators + auto-wait + web-first asserts = far fewer flaky tests.

Data-driven + AI-resilient selectors

Drive one flow with many inputs, and locate elements in ways AI can heal.

EXAMPLE — data-driven Playwright

TEST TECHNIQUE

- › **Data-driven** — one test, a table of inputs & expected results.
- › **Stable hooks** — prefer role/label/test-id over brittle CSS/XPath.
- › **Self-healing** — tools re-find an element when its attributes shift.
- › **Review the heal** — accept the AI's new locator, don't blindly trust it.

```
// one flow, many rows
const cases = [
  { coupon:"SAVE10", total:"90.00" },
  { coupon:"BOGO",   total:"50.00" },
  { coupon:"BAD",    total:"error" } ];
for (const c of cases) {
  test(`coupon ${c.coupon}`, async ({page}) => {
    /* arrange · act · assert on c */
  })
}
```

TIP Locate by intent (role/label/test-id) — that is exactly what self-healing AI keys on.

Why automation is hard

Most automation efforts fail for these reasons — know them before you start.

Flaky tests

timing / order / environment shake trust in the whole suite.

Maintenance cost

UI changes break selectors; the suite rots without care.

Dynamic elements

shifting ids, async loading, animations, iframes, shadow DOM.

Choosing what to automate

not everything pays back; the 100%-automation myth.

Test data & environments

realistic data, setup/teardown, unstable dependencies.

Fragmentation

many browsers · OS versions · devices to cover.

Overcoming the challenges

Each pain point has a known counter-move — many now AI-assisted.

Challenge	How to tackle it
Flaky tests	auto-wait, isolate & reset state, fix root cause (not just retries)
Maintenance	POM/DRY, stable locators, AI self-healing selectors
Dynamic elements	role/test-id locators, explicit waits, web-first assertions
Scope & ROI	risk-based selection, keep exploratory manual, measure ROI
Skills gap	treat tests as code; train testers; pair with developers
Fragmentation	cloud device farms + impact-based selection + parallelism

AI-FIRST self-healing, NL authoring and agentic runners attack maintenance & flakiness — but you still own the assertions.

Where automated tests earn their keep

A suite that isn't in the pipeline is a suite nobody trusts.



KEEP IT FAST parallelise, run unit/API on every PR, reserve heavy UI/e2e for merge or nightly.

GATE a failing automated test blocks the merge — that is the whole point of automating it.

The dos & don'ts of test automation

Most automation fails for process reasons, not tool reasons.

DO

- › Automate stable, high-value, repeated tests.
- › Keep tests independent & self-contained.
- › Use POM/DRY; review it like real code.
- › Run in CI; treat flakes as bugs.

DON'T

- › Chain tests so one failure cascades.
- › Automate an unstable, changing UI.
- › Rely on fixed sleeps & brittle XPath.
- › Chase 100% UI automation.
- › Ignore red — 'just re-run it'.

Metrics for automation health

Track whether your suite is an asset or a liability.

VALUE	HEALTH	EFFICIENCY
› Automation coverage › Requirements covered › Time saved vs manual › Escaped defects	› Flakiness rate › Pass/fail trend › Mean time to repair › Suite run time	› % tests selected (impact) › Parallel speed-up › Maintenance cost / release › Locator-heal rate

GET STARTED

A free stack + an AI-first workflow

Everything below is free / open-source and CI-friendly.

FREE STACK

- › Playwright — web + API + trace viewer
- › Appium — mobile
- › Cucumber — BDD (optional)
- › Healenium — self-healing (Selenium)
- › GitHub Actions — run in CI

AI-FIRST WORKFLOW

- › AI drafts tests from a story / the UI.
- › You review intent & assertions.
- › Run in CI; impact analysis selects the subset.
- › Self-healing fixes broken locators.
- › AI triages failures; you decide the fix.

TAKEAWAY

Automate wisely; let AI do the toil

- Automate stable, repeated, high-value tests.
- Keep UI thin; push checks down the pyramid.
- Regression = run only what a change can break.
- Playwright leads new web projects in 2026.
- **AI self-heals & drafts; you own the assertions.**

READING

Playwright docs

Google Testing Blog

Gartner — Agentic QA

Applitools · Testim blogs

Deliverable idea: a Playwright suite for EShop — POM, data-driven, in CI, with impact-based selection.